SOLID PRINTZIPIOAK

Software Ingeniaritza II

2026-2027

Alain del Ser

Aimar Goñi

Aurkibidea

1. OCP: Open Close Principle..... 3
2. SRP: Single Responsibility Principle..... 4
3. LSP: Liskov Substitution Principle..... 5
4. DIP: Dependency Inversion Principle..... 6
5. ISP: Interface Segregation Principle..... 7

1. OCP: Open Close Principle

   Hasierako kodean, AuthService klaseak if baldintzak erabiltzen zituen autentifikazio-zerbitzua aukeratzeko. Horrek esan nahi zuen zerbitzu berri bat gehitzeko klase horren kodea aldatu behar zela, beste if bat gehituz.

   Aldiz, bigarren kodean, IAuth interfazea sortu dugu, eta autentifikazio-zerbitzu bakoitzak interfaze hori inplementatzen du. Horrela, AuthService klaseak ez du zerbitzu bakoitzaren xehetasunik ezagutu behar, eta zerbitzu berriak gehitu daitezke, klase barri bat sortuz, AuthService klasea aldatu gabe.

   Beraz, errefaktorizazioa da, kodearen barne-egitura hobetu delako, mantengarritasuna eta eskalagarritasuna erraztuz. Errefektarizazio hau egiteko OCP printzipioa aplikatzen da: funtzionalitate berriak gehitu daitezke, lehendik dagoen kodea aldatu beharrik gabe.

2. SRP: Single Responsibility Principle

SRP betetzeko lehen pausua zeregin desberdinak detektatzea izango litzateke, baina hau jada ikusi egiten da erraz, `billDeduction`-aren kalkulua, `VAT`-aren kalkulua eta `billAmount`-ren kalkulua. Beraz zeregina jakinda erraza da zeregin bakoitza klase berri batera eramatea. `DeductionCalculator` eta `VATCalculator` sortu ditugu, azken `billAmount`-aren kalkulua jatorrizko klase/metodoari utziz.
Zereginak sakabanatuak izanda erraza da hauek aldatzea, eta kasu honetan `DeductionCalculator` klasean adierazitako kalkulua gehitu dugu:

```
public float calculate(Bill bill) {
      if (bill.billAmount > 50000) {
          return (bill.billAmount * bill.deductionPercentage + 5) / 100;
      } else {
          return (bill.billAmount * bill.deductionPercentage) / 100;
      }
}
```

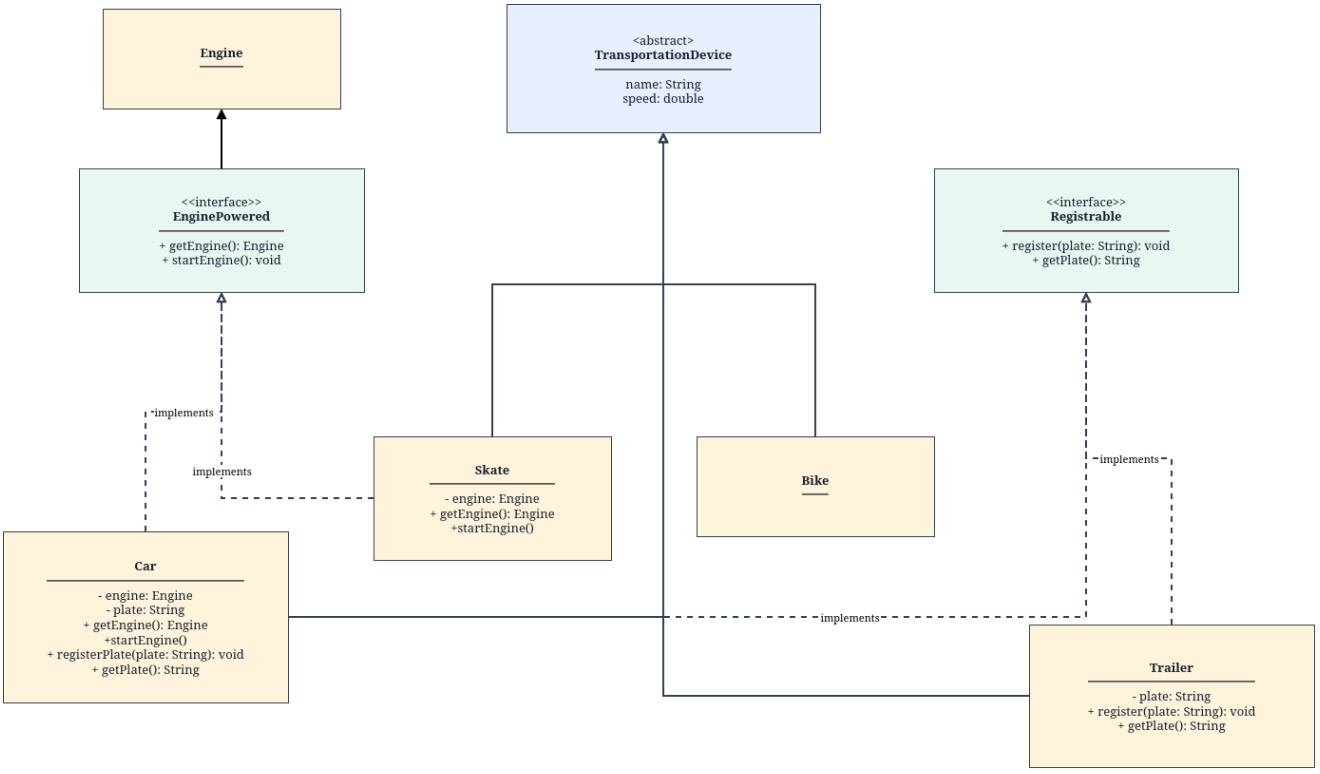
VAT-a %16-tik %18-ra pasatzeko erraza dugu ere aldaketen ondoren. `VATCalculator`-eko klaseko metodoaren balore bat aldatuz zeregina bete dugu.

```
package srp;
public class VATCalculator {
    public float calculate(Bill bill) {
        return bill.billAmount * 0.16f;
    }
}
```

Gure kodean metodoei `Bill` motako aldagaia pasa diegunez gero, 0 kodea (code) duen eta VAT = 0 edo ez ikusteko `VATCalculator` klasean egitea erabaki dugu, VAT kalkuluaren parte dela interpretatu baitugu.

```
package srp;
public class VATCalculator {
    public float calculate(Bill bill) {
        return bill.code.compareTo("0") == 0 ? 0 : bill.billAmount*0.18f;
    }
}
```

3. LSP: Liskov Substitution Principle



1.irudia: LSP UML diagrama.

Arazo hau modelatzeko, 1.irudian ikusten den bezala, ematen diguten `TransportationDevice` klase abstraktutik abitauko gara. Klase honetatik ume guztiak sortuko dira: `Car, Skate, Bike, Trailer`, ... eta ume bakoitza interfaze bat edo beste implementatuko du, beharren arabera. Adibidez `Car` klasea sortu ditugun bi interfazeak inplmentzatzen ditu; interfazeak definitzen dituzten metodoak inplmentatuz gain, bi atributu berri gordetzen ditu ere, bat interfazeko. `Bycicle` klasea aldiz ez du interfazerik inplementazen, klase hutsa da. `Trailer` eta `Skate` klaseak interfaze bana inplementatzen dute barrik, bakoitza dagokion atributu eta metodoekin.

Portaerak interfaze desberdinetan banatzea beharren arabera egokitzeko malgutasuna eskaintzen du; bi interfaze sortuz lau klase abstraktu sortzea sahiesten du (`MotorraMatrikula, EzMotorraMatrikula, MotorraEzMatrikula EzMotorraEzMatrikula`). Hiru portaera nagusi balira zortzi klase abstraktu beharko genituzke, kodearen mantenigarritasuna zailduz.

4. DIP: Dependency Inversion Principle

Ez, kode honek ez du DIP printzipioa betezen, nahiz eta SRP bai aplikatu den. Honen arrazoia da errespontsabilitaeak klase desbderdinetara bidali direla baina gero klase konkretu hautarako kodetzen da eta ez interfazeerako. Interfaze baterako programatzean abstrakzioa lortzen dugu klase egonkorragoak lortuz eta aldaketak egitean kodea ez da ikutu beharko OCP errespetatzea lagunduz. Interfazerako programatzean ziurtatzen gara goi mailako moduluak ez daudela behe mailako moduluen dependentzian; abstrakzioetan oinarritu egon behar dira.

Beraz, implementazio honetan DIP errespetatzeko, `Deduction d=new Deduction();` eta `VAT vat=new VAT();` aldatuko ditugu `IDeductionCalculator` eta `IVATCalculator` interfazeak sortuz. Gero, interfaze hauek inplementatzen hainbat klase sortuko ditugu, bakoitza modu desberdin batean inplementatuz. Bigarren ariketaren jarraituz guk `DeductionCalculatorHigh` / `DeductionCalculatorLow` eta `VAT0, VAT16, VAT18` klaseak sortu ditugu eta aurreko lerroak hauengatik ordezkatu; adibidez `IDeductionCalculator dCalc = new DeductionCalculatorHigh();` eta `IVATCalculator vCalc = new VAT0();`

5. ISP: Interface Segregation Principle

`EmailSender` eta `SMSSender` klaseek `email` eta `telephone` behar dute beraien funtzionalitatea betetzeko, baina `Person` motako objektu bat jasotzen dute. Honek ISP printzipioa bortxatzen duela uste dugu, erabiltzen ez dituzten datu eta metodoekiko (`izena` edo `helbidea`, adibidez) mendekotasuna izatera behartzen dituelako.

Konpontzeko bi interfaze sortu ditugu, bertan beharrezko getter-ak separatuz, `Emailable` eta `SMSable`. `SMSSender` eta `EmailSender` klaseetako metodoetan interfaze hauek sarrera parametro bezala jarriko, gero klase berri bat sortzerkoan, behar dituen interfazeak implementatuko ditu. Horrela javaren poliformismoaz baliatuta, klaseak isntantziatuz, `SMSSender` eta `EmailSender` metodoei pasatuz.

```
public interface Emailable {
	public String getEmail();
}
public class EmailSender {
	public static void sendEmail(Emailable c, String message){
		// Mezu bat bidaltzen du Person klaseko korreo helbidera.
		IO.println("EmailSender exekutatuta: " + message);}
```

Goikoa ongi dagoela frogatzeko `GmailAccount` sortu dugun `name` eta `emailAddress` atributuekin. Klasea `Emailable` implementatzen duela ipini dugu azaldutakoarengatik.
`GmailAccountTest` klasean sortu dugu main metodo bat aurreko guztia probatzeko. Esan bezala, proban, `GmailAccount` instantziatzearekin eta `sendEmail()` metodoari pasatzearekin nahiko da.

```
public class GmailAccount implements Emailable {
	String name, emailAddress;
	public GmailAccount(String name, String emailAddress) {
		this.name = name;
		this.emailAddress = emailAddress;
	}
	@Override
	public String getEmail() {
		return emailAddress;
	}
}
public class GmailAccountTest {
	void main() {
	     GmailAccountkontua=new GmailAccount("user1","user1@gmail.com");
	     EmailSender.sendEmail(kontua,"PROBA MEZUA");
	}
}
```